

Fundamentos de Teoría de la Computación

Resumen teorías - Parte 2 - Lógica e Inteligencia Artificial

Clase 8 - Lógica de enunciados o proposicional	5
¿Qué es la lógica?	5
¿Qué es el razonamiento?	5
El lenguaje de la lógica	5
El lenguaje simbólico de la lógica	7
Sintaxis	7
Alfabeto	7
Gramática	8
Semántica	9
Negación	9
Conjunción	9
Disyunción	10
Condicional	10
Bicondicional	11
Tablas de verdad para enunciados compuestos	12
Razonamiento	12
Premisas verdaderas y conclusión verdadera	13
Razonamiento correcto	13
Razonamiento incorrecto	13
Premisas falsas y conclusión falsa	14
Razonamiento correcto	14
Razonamiento incorrecto	14
Premisas falsas y conclusión verdadera	15
Razonamiento correcto	15
Razonamiento incorrecto	15
Premisas verdaderas y conclusión falsa	16
Razonamiento correcto	16
Razonamiento incorrecto	16
Resumen de razonamiento	16
Argumentación	17
Clase 9 - Lógica de enunciados parte 2	18
Interpretación y satisfacción	18
Tautología y contradicción	19
Equivalencias lógicas	19
Implicación lógica y equivalencia lógica	20
Conjuntos adecuados de conectivas	21

Formas normales	22
Nor	22
Nand	22
Argumentaciones	23
Clase 10 - Cálculo de enunciados L	24
Mecanismos formales de razonamiento	24
El Sistema formal L	24
Axiomas de L	24
Reglas de inferencia de L	25
Demostración o derivación en L	25
Ejemplo	26
Teorema de L	26
Metateorema de la Deducción	27
Corrección (sensatez) y completitud de un sistema deductivo	27
$\vdash_L A$	27
$\models A$	28
Corrección del sistema deductivo L	28
Consistencia	28
L es consistente	28
Decidibilidad	29
Clase 11 - Lógica de predicados	30
Predicados	30
Cuantificadores	31
Relación entre $\forall$ y $\exists$	31
Sintaxis	31
Alfabeto	32
Scope de los cuantificadores	33
Clase 12 - Lógica de predicados - Semántica	34
Dominio de interpretación	34
Interpretación	36
Semántica	37
Valoración	38
Satisfacción	39
i-equivalencias	40
Verdad y validez	40
Clase 13 - Inteligencia artificial	43
Razones de éxito	43
¿Mejoró el hardware?	43
¿Se inventaron nuevas estructuras de datos?	43
¿Se diseñaron nuevos algoritmos?	44
¿Son IA todos los sistemas computacionales que imitan nuestras funciones cognitivas?	44
Machine Learning (máquinas que aprenden)	44
ML con ANN	45

Parámetros e hiper parámetros	47
Métodos de ajuste de hiper parámetros	48
Ciclo de vida del software tradicional vs ML	49
Conclusiones	49
Clase 14 - Verificación axiomática de programas	51
¿Cómo probar un programa?	51
¿Cómo verificar un programa?	51
Definiciones	52
Componentes del método de verificación axiomática (Lógica de Hoare)	53
Lenguaje de programación (programas con while):	53
Lenguaje de especificación (lógica de predicados):	54
Axiomática	54
Axioma de la asignación (ASI)	54
Regla de la secuencia (SEC)	55
Regla del condicional (COND)	55
Regla de la repetición (REP)	55
Regla de consecuencia (CONS)	56
Composicionalidad	56
Especificaciones	57
Sensatez, completitud y automatización de la verificación	58
Sensatez	58
Completitud	58
Automatización	58

Clase 8 - Lógica de enunciados o proposicional

¿Qué es la lógica?

Trata acerca de los medios a través de los cuales puede propagarse y articularse el conocimiento

Es la disciplina que estudia las formas de razonamiento

¿Qué es el razonamiento?

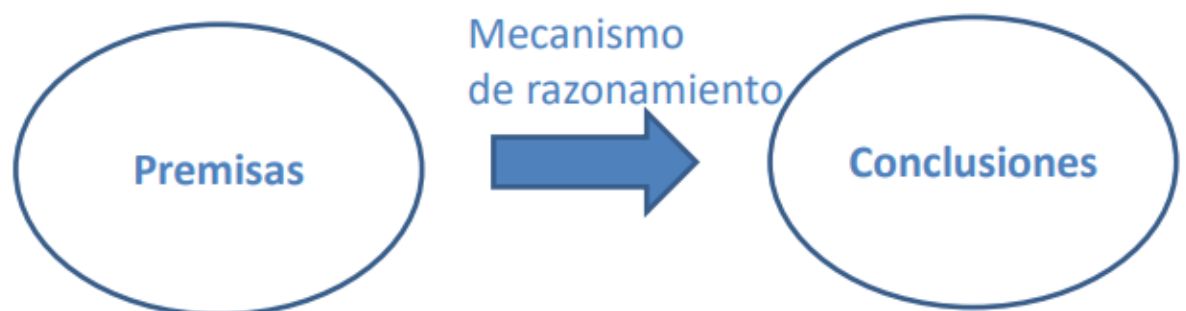
Se apoya en verdades supuestas, para obtener otras como resultado de la actividad de razonamiento

Sirve para justificar ciertas verdades, si son consecuencia (formal) de conocimientos previamente aceptados:

- Premisas del razonamiento (propuestas)
- Conclusión (conocimiento nuevo)

Los conocimientos se expresan mediante proposiciones o enunciados (lenguaje de la lógica)

Razonamiento = encadenamiento de enunciados formado por premisas y conclusiones



El lenguaje de la lógica

La lógica proposicional, también conocida como lógica de enunciados, es un sistema formal cuyos elementos representan proposiciones o enunciados

Esta lógica no tiene, por sí misma, mucha utilidad para la representación del conocimiento. Está justificado detenerse en ella porque permite introducir de una manera sencilla algunos conceptos que, explicados directamente para la lógica de predicados, son más difíciles de captar

Nos interesa examinar los mecanismos de razonamiento con precisión matemática. Esta precisión requiere que el lenguaje que usemos no dé lugar a confusiones, lo cual conseguimos mediante un lenguaje simbólico donde cada símbolo tenga un significado bien definido.

Dada una frase en lenguaje natural, en primer lugar, podemos observar si se trata de una frase simple o de una frase compuesta

Una frase simple consta de un sujeto y un predicado. Por ejemplo:

- Java es un lenguaje de programación
- Android es un sistema operativo moderno

Una frase compuesta se forma a partir de frases simples por medio de algún término de enlace (o conectiva). Por ejemplo:

- Java es un lenguaje de programación y Java es compatible con Android
- Si Android es un sistema operativo moderno entonces Android soporta Java

En segundo lugar, vamos a suponer que todas las frases simples pueden ser verdaderas o falsas

Ahora bien, en castellano hay frases que no son ni verdaderas ni falsas, por tanto tenemos que usar un término diferente

Hablaremos de enunciados (o proposiciones) para referirnos a frases que son verdaderas o falsas

Y distinguiremos entre enunciados simples (atómicos) o enunciados compuestos

Denotamos los enunciados con letras mayúsculas A, B, C, ...

Para construir enunciados compuestos introducimos símbolos para las conectivas

Las conectivas más comunes y los símbolos que emplearemos para denotarlas son los siguientes:

- $\neg A$: negación de A
- $A \wedge B$: conjunción de A y B
- $A \vee B$: disyunción de A o B
- $A \rightarrow B$: si A entonces B
- $A \leftrightarrow B$: A si y sólo si B

Así, los enunciados compuestos vistos antes:

- Java es un lenguaje de programación y Java es compatible con Android
- Si Android es un sistema operativo moderno entonces Android soporta Java

Pueden escribirse simbólicamente de la siguiente forma: $A \wedge B$ y $C \rightarrow D$

- A simboliza “Java es un lenguaje de programación”
- B simboliza “Java es compatible con Android”
- C simboliza “Android es un sistema operativo moderno”
- D simboliza “Android soporta Java”

El lenguaje simbólico de la lógica

Para estudiar los principios del razonamiento, necesitamos:

- Capturar y formalizar las estructuras del lenguaje natural en un lenguaje simbólico
- Para luego formalizar los mecanismos de razonamiento que se aplican sobre dichas estructuras lingüísticas

El lenguaje tiene sintaxis y semántica

Sintaxis

El lenguaje simbólico consta de un conjunto de:

- Símbolos primitivos (el alfabeto o vocabulario)
- Un conjunto de reglas de formación (la gramática) que nos dice cómo construir fórmulas bien formadas a partir de los símbolos primitivos

Alfabeto

El alfabeto de un sistema formal es el conjunto de símbolos que pertenecen al lenguaje del sistema

Si L es el nombre del sistema de lógica proposicional, entonces el alfabeto de L consiste en:

- Una cantidad finita pero arbitrariamente grande de variables proposicionales (o variables de enunciado). En general se las toma del alfabeto latino, empezando por la letra p , luego q , r , etc., y utilizando subíndices cuando es

necesario. Las variables de enunciado representan enunciados simples como "está lloviendo" o «java es un lenguaje de programación»

- Un conjunto de operadores lógicos o conectivas: $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$
- Dos signos de puntuación: el paréntesis izquierdo y el paréntesis derecho

Gramática

Una vez definido el alfabeto, el siguiente paso es determinar qué combinaciones de símbolos pertenecen al lenguaje del sistema

Esto se logra mediante una gramática formal

La misma consiste en un conjunto de reglas que definen recursivamente las cadenas de caracteres que pertenecen al lenguaje. A las cadenas de caracteres construidas según estas reglas se las llama fórmulas bien formadas (fbf), y también se las conoce como formas enunciativas

Las reglas del sistema L son:

- Las variables de enunciado del alfabeto de L son formas enunciativas. Es decir, p, q, ...
- Si A y B son formas enunciativas de L, entonces también lo son $(\neg A)$, $(A \wedge B)$, $(A \vee B)$, $(A \rightarrow B)$ y $(A \leftrightarrow B)$
- Sólo las expresiones que pueden ser generadas mediante las cláusulas i y ii en un número finito de pasos son formas enunciativas de L

Según estas reglas, las siguientes cadenas de caracteres son ejemplos de formas enunciativas:

p, q, r, por la definición (i).

$(\neg p)$, $(q \wedge r)$, $(p \rightarrow q)$, por la definición (ii) y la línea anterior.

$((\neg p) \vee (q \wedge r)) \rightarrow (p \rightarrow q)$, por la definición (ii) y la línea anterior.

Y los siguientes son ejemplos de fórmulas que no son formas enunciativas:

Fórmula	Error	Corrección
(p)	Sobran paréntesis	p
$\neg (p)$	Mal los paréntesis	$(\neg p)$
$p \rightarrow q$	Faltan paréntesis	$(p \rightarrow q)$

Semántica

Como todo enunciado simple es verdadero o falso, una variable de enunciado tomará uno u otro valor de verdad: V (verdadero) o F (falso)

La verdad o falsedad de un enunciado compuesto depende de la verdad o falsedad de los enunciados simples que lo constituyen, y de la forma en que están conectados

Primeramente vamos a analizar el significado de cada una de las conectivas, mediante tablas de verdad

Negación

Sea A un enunciado

Denotaremos con $\neg A$ a su negación. Si A es verdadero entonces $\neg A$ es falso, y recíprocamente si A es falso entonces $\neg A$ es verdadero

La siguiente es la tabla de verdad que especifica el significado de esta conectiva:

A	$\neg A$
V	F
F	V

La conectiva $\neg$ da lugar a una función de verdad llamada $f_{\neg}$ que tiene como dominio y codominio al conjunto $\{V, F\}$ y se define así:

$$\begin{aligned}f_{\neg}(V) &= F \\f_{\neg}(F) &= V\end{aligned}$$

Conjunción

Sean A y B dos enunciados, denotamos con $A \wedge B$ a la conjunción de ambos

Su tabla de verdad es la siguiente:

A	B	$A \wedge B$
V	V	V
V	F	F
F	V	F
F	F	F

La conectiva $\wedge$ define una función de verdad $f^\wedge$ de dos argumentos:

$$f^\wedge (V, V) = V$$

$$f^\wedge (V, F) = F$$

$$f^\wedge (F, V) = F$$

$$f^\wedge (F, F) = F$$

Disyunción

Sean A y B dos enunciados. En castellano tenemos dos usos distintos para la disyunción “o”. Elegimos “A o B o ambos”, que denotamos con $A \vee B$

Su tabla de verdad es:

A	B	$A \vee B$
V	V	V
V	F	V
F	V	V
F	F	F

La conectiva $\vee$ define una función de verdad $f^\vee$ de dos argumentos, como la anterior:

$$f^\vee (V, V) = V$$

$$f^\vee (V, F) = V$$

$$f^\vee (F, V) = V$$

$$f^\vee (F, F) = F$$

Nótese que el otro uso de la disyunción, es decir “A o B pero no a mbos”, se puede simbolizar mediante $(A \vee B) \wedge \neg(A \wedge B)$

Condicional

Sean A y B dos enunciados

Utilizaremos $A \rightarrow B$ para representar el enunciado “A implica a B” o “si A entonces B”

En este caso el significado intuitivo de esta frase genera algunos conflictos con su significado formal

La tabla de verdad de esta conectiva es la siguiente:

A	B	$A \rightarrow B$
V	V	V
V	F	F
F	V	V
F	F	V

De la misma forma que las anteriores, la conectiva $\rightarrow$ define una función de verdad de dos argumentos:

$$f \rightarrow (V, V) = V$$

$$f \rightarrow (V, F) = F$$

$$f \rightarrow (F, V) = V$$

$$f \rightarrow (F, F) = V$$

Bicondicional

Sean A y B dos enunciados. Denotamos el enunciado “A si y sólo si B” o “A equivale a B” con $A \leftrightarrow B$

Este enunciado será verdadero cuando A y B tengan el mismo valor de verdad (ambos verdaderos o ambos falsos), y sólo en dicho caso

La tabla de verdad es la siguiente:

A	B	$A \leftrightarrow B$
V	V	V
V	F	F
F	V	F
F	F	V

Como antes, la conectiva $\leftrightarrow$ define una función de verdad de dos argumentos:

$$f \leftrightarrow (V, V) = V$$

$$f \leftrightarrow (V, F) = F$$

$$f \leftrightarrow (F, V) = F$$

$$f \leftrightarrow (F, F) = V$$

Tablas de verdad para enunciados compuestos

En lo que sigue veremos que el valor de verdad de un enunciado compuesto depende de los valores de verdad de los enunciados simples que lo forman, aplicando las funciones de verdad de las conectivas

La tabla es una representación gráfica de una función de verdad, cuyo número de argumentos es igual al número de variables distintas que intervienen:

p	q	r	$(q \wedge r)$	$p \rightarrow (q \wedge r)$
V	V	V	V	V
V	V	F	F	F
V	F	V	F	F
V	F	F	F	F
F	V	V	V	V
F	V	F	F	V
F	F	V	F	V
F	F	F	F	V

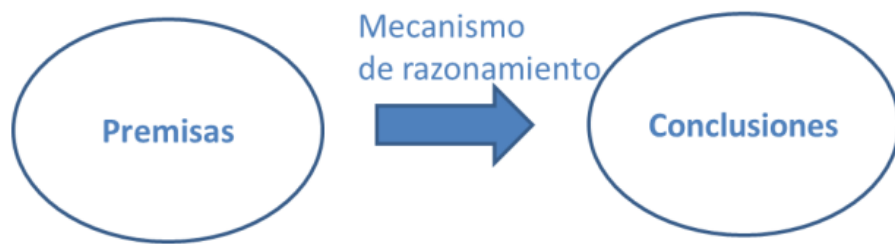
3 letras

$2^3 = 8$ filas

A una forma enunciativa con n variables diferentes, le corresponde una función de verdad de n argumentos, y la tabla de verdad tendrá 2^n filas, una para cada una de las posibles combinaciones de valores de verdad

Razonamiento

Usaremos este lenguaje para analizar los razonamientos:



El razonamiento puede ser:

- Correcto o válido: si la manera en que está construido garantiza la conservación de la verdad. Si las premisas son V, entonces la conclusión es necesariamente V
- Incorrecto o inválido: su construcción es defectuosa (no hay garantía acerca del valor de verdad de la conclusión)

Deducción = razonamiento correcto

Premisas verdaderas y conclusión verdadera

Razonamiento correcto

- Si Juan es mendocino entonces Juan es argentino V
- Si Juan es argentino entonces Juan es sudamericano V
- Por lo tanto: si Juan es mendocino entonces Juan es sudamericano V

Este razonamiento posee la siguiente estructura lógica:

- Si A entonces B V
- Si B entonces C V
- Por lo tanto: si A entonces C V

A esta forma de razonamiento se la conoce como silogismo hipotético

Razonamiento incorrecto

- Si Juan es mendocino entonces Juan es sudamericano V
- Si Juan es argentino entonces Juan es sudamericano V
- Por lo tanto: si Juan es mendocino entonces Juan es argentino V

Este razonamiento posee la siguiente estructura de razonamiento:

- Si A entonces B V
- Si C entonces B V
- Por lo tanto: si A entonces C V

Esta estructura evidencia una forma incorrecta de razonar, que en este caso permitió obtener una conclusión verdadera a partir de premisas verdaderas

Sin embargo la misma estructura de razonamiento podría instanciarse con otros enunciados que dejarían en evidencia su incorrección

Premisas falsas y conclusión falsa

Razonamiento correcto

- Si Juan es argentino entonces Juan es africano F
- Si Juan es africano entonces Juan es asiático F
- Por lo tanto: si Juan es argentino entonces Juan es asiático F

Este razonamiento posee la siguiente estructura lógica:

- Si A entonces B F
- Si B entonces C F
- Por lo tanto: si A entonces C F

Observamos que se trata de un razonamiento correcto (nuevamente el esquema del silogismo), que nos ha permitido deducir información falsa a partir de premisas falsas

Razonamiento incorrecto

- Si Juan es chino entonces Juan es sudamericano F
- Si Juan es peruano entonces Juan es sudamericano F
- Por lo tanto: si Juan es chino entonces Juan es peruano F

Este razonamiento posee la siguiente estructura de razonamiento:

- Si A entonces B F
- Si C entonces B F

- Por lo tanto: si A entonces C F

También esta forma de razonamiento incorrecto nos ha permitido deducir información falsa a partir de premisas falsas

Premisas falsas y conclusión verdadera

Razonamiento correcto

Es posible partir de premisas falsas y arribar a conclusiones verdaderas

Podríamos decir que se llega a la verdad “por casualidad”

Veamos el siguiente ejemplo que nuevamente aplica el silogismo como esquema de razonamiento correcto:

- Si Juan es argentino entonces Juan es africano F
- Si Juan es africano entonces Juan es sudamericano F
- Por lo tanto: si Juan es argentino entonces Juan es sudamericano V

Este razonamiento posee la siguiente estructura lógica:

- Si A entonces B F
- Si B entonces C F
- Por lo tanto: si A entonces C V

Razonamiento incorrecto

Una situación similar ocurre si aplicamos una forma incorrecta de razonar, como en el siguiente ejemplo:

- Si Juan es mendocino entonces Juan es africano F
- Si Juan es argentino entonces Juan es africano F
- Por lo tanto: si Juan es mendocino entonces Juan es argentino V

Este razonamiento posee la siguiente estructura de razonamiento:

- Si A entonces B F
- Si C entonces B F
- Por lo tanto: si A entonces C V

Premisas verdaderas y conclusión falsa

Razonamiento correcto

El razonamiento correcto preserva la verdad, no es posible partir de premisas verdaderas y llegar a conclusiones falsas a través de un razonamiento correcto

Razonamiento incorrecto

Esta situación puede ocurrir únicamente si aplicamos un razonamiento incorrecto, como en el siguiente ejemplo:

- Si Juan es mendocino entonces Juan es argentino V
- Si Juan es salteño entonces Juan es argentino V
- Por lo tanto: si Juan es mendocino entonces Juan es salteño F

Este razonamiento posee la siguiente estructura de razonamiento:

- Si A entonces B V
- Si C entonces B V
- Por lo tanto: si A entonces C F

Resumen de razonamiento

En resumen, un razonamiento es directamente incorrecto cuando a partir de premisas verdaderas permite arribar a una conclusión falsa

O bien es incorrecto porque tiene la estructura de un razonamiento incorrecto (aunque la conclusión sea verdadera)

La corrección de la forma solamente garantiza que si las premisas son verdaderas entonces lo será también la conclusión

El caso $F \rightarrow V$ es de gran importancia en el método científico, ya que permite razonar correctamente, pero sobre hipótesis que podrían ser falsas

La verdad de la conclusión no nos asegura nada acerca de la verdad de las premisas

Argumentación

Una forma argumentativa es una sucesión finita de formas enunciativas, de las cuales la última se considera como la conclusión de las anteriores, conocidas como premisas

La notación es: $A_1, A_2, \dots, A_n \therefore A$

Para que una forma argumentativa sea válida debe representar un razonamiento correcto. Es decir, bajo cualquier asignación de valores de verdad a las variables de enunciado, si las premisas $A_1, A_2, \dots, A_n$, toman el valor V, la conclusión A también debe tomar el valor V

Una forma argumentativa $A_1, A_2, \dots, A_n \therefore A$ es inválida si es posible asignar valores de verdad a las variables de enunciado que aparecen en ella, de tal manera que $A_1, A_2, \dots, A_n$, tomen el valor V y A tome el valor F. De lo contrario la forma argumentativa es válida

Clase 9 - Lógica de enunciados parte 2

Interpretación y satisfacción

Una interpretación es una función que relaciona los elementos de los dominios sintáctico y semántico de la lógica considerada



En el caso particular de la lógica proposicional, una interpretación I consiste en una función de valuación v que asigna a cada variable de enunciado el valor de verdad V o F

Siendo $P = \{p_1, p_2, \dots\}$ el conjunto de variables de enunciado:

$$v : P \rightarrow \{V, F\}$$

Para extender el dominio de la función de valuación de las variables de enunciado a las fbfs en general, se define una regla semántica para cada una de las reglas sintácticas de la gramática:

$\models_v p$ si y sólo si $v(p) = V$

$\models_v (\neg A)$ si y sólo si no es el caso que $\models_v A$

$\models_v (A \vee B)$ si y sólo si o bien $\models_v A$ o bien $\models_v B$ o ambos

$\models_v (A \wedge B)$ si y sólo si $\models_v A$ y $\models_v B$

$\models_v (A \rightarrow B)$ si y sólo si no es el caso que $\models_v A$ y no $\models_v B$

$\models_v (A \leftrightarrow B)$ si y sólo si $\models_v (A \rightarrow B)$ y $\models_v (B \rightarrow A)$

También lo puedo escribir así: $v(A \wedge B) = V$ si y sólo si $v(A) = V$ y $v(B) = V$

Tautología y contradicción

Una forma enunciativa es una tautología si siempre toma el valor de verdad V, considerando todas y cada una de las posibles asignaciones de valores de verdad a las variables de enunciado que contiene

Si en cambio siempre toma el valor de verdad F, la forma enunciativa se conoce como contradicción

El método para determinar si una forma enunciativa es una tautología o una contradicción consiste en construir su tabla de verdad

Por ejemplo:

- $(p \vee (\neg p))$ es una tautología
- $(p \wedge (\neg p))$ es una contradicción
- $(p \vee q)$ no es ni una tautología ni una contradicción

Claramente, toda tautología con n variables tiene asociada una misma función de verdad de n argumentos, es decir la misma tabla de verdad de 2^n filas donde la última columna siempre contiene el valor V. Una situación similar ocurre para las contradicciones con el valor F

Equivalencias lógicas

Notemos que existen 2^{2^n} funciones de verdad distintas de n argumentos, que corresponden a las 2^{2^n} maneras posibles de disponer los valores V y F en la última columna de una tabla de verdad de 2^n filas

p	q	r	?
V	V	V	F
V	V	F	V
V	F	V	V
V	F	F	V
F	V	V	V
F	V	F	V
F	F	V	V
F	F	F	V

$(\sim p \vee \sim q \vee \sim r)$

$\sim(p \wedge q \wedge r)$

$\sim\sim(\sim p \vee \sim q \vee \sim r)$

INFINITAS MAS!!!

Tendremos mas de una forma enunciativa asociadas a una misma función de verdad

Implicación lógica y equivalencia lógica

Sean A y B dos enunciados

Diremos que “A es lógicamente equivalente a B” (lo denotaremos con $A \Leftrightarrow B$) si la forma enunciativa $A \leftrightarrow B$ es una tautología

Por ejemplo: $\neg(p \vee q)$ es lógicamente equivalente a $((\neg p) \wedge (\neg q))$

También diremos que “A implica lógicamente a B” o que “B es consecuencia lógica de A” (lo denotaremos con $A \Rightarrow B$) si la forma enunciativa $A \rightarrow B$ es una tautología

Por ejemplo: $(p \wedge q)$ implica lógicamente a p

Las siguientes son equivalencias lógicas muy conocidas, por resultar útiles a la hora de manipular formas enunciativas:

Ley de Doble Negación

$$\neg (\neg p) \Leftrightarrow p$$

Ley Conmutativa de la Conjunción

$$p \wedge q \Leftrightarrow q \wedge p$$

Ley Conmutativa de la Disyunción

$$p \vee q \Leftrightarrow q \vee p$$

Ley Asociativa de la Conjunción

$$(p \wedge q) \wedge r \Leftrightarrow p \wedge (q \wedge r)$$

Ley Asociativa de la Disyunción

$$(p \vee q) \vee r \Leftrightarrow p \vee (q \vee r)$$

Leyes de De Morgan

$$\neg (p \vee q) \Leftrightarrow (\neg p) \wedge (\neg q)$$

$$\neg (p \wedge q) \Leftrightarrow (\neg p) \vee (\neg q)$$

Leyes de Distribución

$$p \wedge (q \vee r) \Leftrightarrow (p \wedge q) \vee (p \wedge r)$$

$$p \vee (q \wedge r) \Leftrightarrow (p \vee q) \wedge (p \vee r)$$

Leyes de Absorción

$$p \wedge p \Leftrightarrow p$$

$$p \vee p \Leftrightarrow p$$

Conjuntos adecuados de conectivas

Un conjunto adecuado de conectivas es un conjunto tal que toda función de verdad puede representarse por medio de una forma enunciativa en la que sólo aparezcan conectivas de dicho conjunto

Los pares $\{\neg, \wedge\}$, $\{\neg, \vee\}$ y $\{\neg, \rightarrow\}$ son conjuntos adecuados de conectivas

$$(A \vee B) \quad \text{equivalente a} \quad \neg (\neg A \wedge \neg B)$$

$$(A \wedge B) \quad \text{equivalente a} \quad \neg (\neg A \vee \neg B)$$

$$(A \rightarrow B) \quad \text{equivalente a} \quad \neg (A \wedge \neg B)$$

$$(A \rightarrow B) \quad \text{equivalente a} \quad \neg A \vee B$$

$$(A \rightarrow B) \quad \text{equivalente a} \quad \neg (A \wedge \neg B)$$

Formas normales

Hemos visto que a partir de toda forma enunciativa puede construirse una tabla de verdad

Vamos a formular ahora un resultado en un sentido recíproco

Proposición: toda función de verdad es la función de verdad determinada por una forma enunciativa restringida. Llamamos forma enunciativa restringida a una forma enunciativa en la que solamente figuran las conectivas $\neg$, $\wedge$, $\vee$

Corolario: toda forma enunciativa, que no es una contradicción, es lógicamente equivalente a una forma enunciativa restringida de la forma:

$$\bigvee_{i=1}^m \bigwedge_{j=1}^n Q_{ij}$$

Donde cada Q_{ij} es una variable de enunciado o la negación de una variable de enunciado

Esta forma se denomina forma normal disyuntiva

Nor

Se denota con $\downarrow$ y no es más que la negación de la disyunción, es decir $\neg(p \vee q)$

Su tabla de verdad es por lo tanto la siguiente:

p	q	p $\downarrow$ q
V	V	F
V	F	F
F	V	F
F	F	V

Nand

Se denota con $|$ y no es más que la negación de la conjunción, es decir $\neg(p \wedge q)$

Proposición: los conjuntos unitarios $\{\downarrow\}$ y $\{\downarrow\}$ son conjuntos adecuados de conectivas: toda función de verdad puede expresarse mediante una forma enunciativa en la que sólo aparece la conectiva $\downarrow$, o sólo aparece la conectiva $\downarrow$

Argumentaciones

Proposición: la forma argumentativa $A_1, A_2, \dots, A_n$ therefore A es válida si y sólo si la forma enunciativa $(A_1 \wedge A_2 \wedge \dots \wedge A_n) \rightarrow A$ es una tautología (es decir, si y sólo si la conjunción de las premisas implican lógicamente a la conclusión)

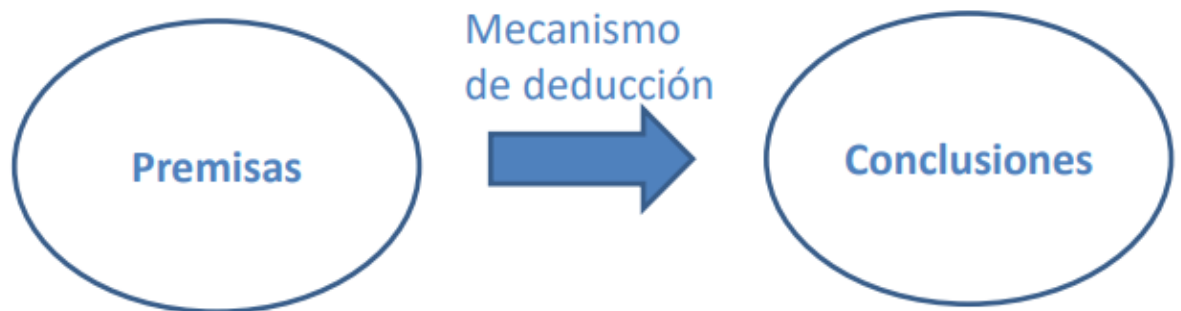
Para referirnos a formas argumentativas válidas utilizamos la siguiente notación:

$\Gamma \models A$ que se lee: “ Γ implica lógicamente a A ” o “ A se deduce de Γ ”, siendo Γ un conjunto de premisas. $\Gamma = \{A_1, A_2, \dots, A_n\}$

Clase 10 - Cálculo de enunciados L

Mecanismos formales de razonamiento

Un mecanismo formal de razonamiento (o de inferencia, deducción, demostración) consiste en una colección de reglas que pueden ser aplicadas sobre cierta información inicial para derivar información adicional, en una forma puramente sintáctica



A continuación se presentan un sistema deductivo llamado L

El Sistema formal L

Un sistema deductivo axiomático está compuesto por:

- Un conjunto de axiomas (en realidad esquemas de axiomas)
- Un conjunto de reglas de inferencia

Los axiomas son fórmulas bien formadas

Las reglas determinan qué fórmulas pueden inferirse a partir de qué fórmulas

Axiomas de L

Los axiomas de un sistema axiomático son un conjunto de fórmulas que se toman como punto de partida para las demostraciones

Un conjunto de axiomas muy conocido para la lógica proposicional es el que definió J. Łukasiewicz:

- L1: $(A \rightarrow (B \rightarrow A))$
- L2: $(A \rightarrow (B \rightarrow C)) \rightarrow ((A \rightarrow B) \rightarrow (A \rightarrow C))$

- L3: $((\neg A) \rightarrow (\neg B)) \rightarrow (B \rightarrow A)$

Reglas de inferencia de L

El sistema L tiene una única regla de inferencia, el modus ponens:

MP: a partir de A y de $(A \rightarrow B)$ se infiere B

Una regla de inferencia es una función que asigna una fórmula (conclusión) a un conjunto de fórmulas (premisas)

Demostración o derivación en L

Una demostración es una sucesión de aplicaciones de reglas de inferencia que permite llegar a una conclusión a partir de determinadas premisas y axiomas

Como la derivación es una relación transitiva, todas las fórmulas que se vayan obteniendo sucesivamente están implicadas por las premisas o axiomas



La notación es la siguiente $\Gamma \vdash_L A_n$

Se lee: “A partir de las premisas contenidas en el conjunto Γ (Gamma) se deduce (o infiere o deriva) la conclusión A_n ”

Γ es un conjunto de premisas (o hipótesis)

A_n es la conclusión a la que se quiere llegar

Tanto las premisas como la conclusión son fórmulas bien formadas (fbf), es decir, son sentencias escritas en el lenguaje de la lógica

Una demostración es una sucesión finita de fórmulas bien formadas $A_1, A_2, \dots, A_n$, tal que para todo i , con $1 \leq i \leq n$, A_i es una premisa o es un axioma, o bien se infiere de miembros anteriores de la sucesión como consecuencia directa de la aplicación de una regla de inferencia (MP)

Ejemplo

El esquema general es este:

$$\Gamma \vdash_L A_n$$

Nuestro ejemplo es: $\{((p \rightarrow q) \rightarrow r), q\} \vdash_L (p \rightarrow r)$

Para demostrar que la conclusión realmente se deduce de esas premisas hay que armar una demostración

Vamos a construir la demostración:

1.	$q \rightarrow (p \rightarrow q)$	instancia de L1
2.	q	hipótesis
3.	$(p \rightarrow q)$	aplicación de MP entre 1 y 2
4.	$((p \rightarrow q) \rightarrow r)$	hipótesis
5.	r	aplicación de MP entre 3 y 4
6.	$r \rightarrow (p \rightarrow r)$	instancia de L1
7.	$(p \rightarrow r)$	aplicación de MP entre 5 y 6

Se observa que construimos una cadena de 7 pasos, $A_1, A_2, \dots, A_7$, donde el último paso es la fbf que queríamos derivar

En cada uno de los pasos usamos:

- Una premisa (paso 2 y paso 4)
- Un axioma (paso 1 y 6)
- La aplicación de la regla de inferencia MP (paso 3, 5 y 7)

Esta cadena DEMUESTRA que efectivamente a partir de esas 2 premisas, sean A_2 , y A_4 se deriva la conclusión $A_7 (p \rightarrow r)$

Teorema de L

Cuando una fbf se puede demostrar a partir del conjunto vacío de hipótesis se la llama teorema de L

Por ejemplo $(p \rightarrow p)$ es un teorema de L

La notación es $\vdash_L (p \rightarrow p)$

Vean que Gamma no aparece (porque está vacío)

Para demostrar que $(p \rightarrow p)$ es un teorema de L tenemos que construir la cadena que termine en $(p \rightarrow p)$, por ejemplo la siguiente cadena es una demostración

- | | | |
|----|---|------------------------------|
| 1. | $(p \rightarrow ((p \rightarrow p) \rightarrow p)) \rightarrow ((p \rightarrow (p \rightarrow p)) \rightarrow (p \rightarrow p))$ | instanciando el axioma L_2 |
| 2. | $p \rightarrow ((p \rightarrow p) \rightarrow p)$ | instanciando el axioma L_1 |
| 3. | $(p \rightarrow (p \rightarrow p)) \rightarrow (p \rightarrow p)$ | MP entre 1 y 2 |
| 4. | $p \rightarrow (p \rightarrow p)$ | instanciando el axioma L_1 |
| 5. | $(p \rightarrow p)$ | MP entre 3 y 4 |

Metateorema de la Deducción

Si $\Gamma \cup \{A\} \vdash_L B$ entonces $\Gamma \vdash_L (A \rightarrow B)$

Hemos demostrado $\Gamma \vdash_L (p \rightarrow p)$ para Γ conjunto vacío

Aplicando el metateorema de la deducción lo podemos resolver más fácil

Si demuestro $\Gamma \cup \{p\} \vdash_L p$ entonces por el MT de la Deducción obtengo $\Gamma \vdash_L (p \rightarrow p)$

Cuántos pasos tiene la demostración de $\Gamma \cup \{p\} \vdash_L p$? 1 solo

Corrección (sensatez) y completitud de un sistema deductivo

De un sistema deductivo se espera naturalmente que:

- Sus demostraciones produzcan conclusiones correctas
- Y también en lo posible la propiedad inversa, es decir que sea capaz de demostrar todas y cada una de ellas

Formalmente ambas propiedades se pueden formular de la siguiente manera:

- El sistema deductivo L es correcto (o sensato) si $\vdash_L A$ implica $\models A$. Si A es teorema, entonces A es tautología
- Recíprocamente, un sistema deductivo L es completo si $\models A$ implica $\vdash_L A$. Si A es tautología entonces A es teorema

Notemos las diferencias entre $\vdash_L A$ y $\models A$

$\vdash_L A$

A es teorema, lo demuestro construyendo su demostración (una secuencia $A_1, A_2, \dots, A$, tal que para todo i , con $1 \leq i \leq n$, A_i es un axioma, o bien se infiere de miembros anteriores de la sucesión por MP)

Es un procedimiento sintáctico, hago «cuentas» en el cálculo

$\models A$

A es tautología, lo demuestro construyendo su tabla de verdad y comprobando que da Verdadero en todas las filas

Es un procedimiento «semántico», aplico las funciones de verdad

Corrección del sistema deductivo L

La corrección de un sistema deductivo está garantizada si se cuenta con axiomas verdaderos y reglas de inferencia correctas (o sensatas), es decir que preservan la verdad

Entonces ¿Cómo demostramos que L es correcto?

Debemos demostrar lo siguiente:

- Sus axiomas L1, L2 y L3 son tautologías
- Su regla MP preserva la verdad

El sistema L es sensato (ambas cosas se prueban, una por tabla de verdad y la otra por ej, por el absurdo): si $\vdash_L A$ entonces $\models A$, es decir, si A es teorema de L entonces A es tautología

Consistencia

Un conjunto de fbf, sea G, no es consistente cuando existe alguna fbf A tal que: $G \vdash_L A$ y $G \vdash_L (\neg A)$

Es decir, G permite deducir un enunciado y su negación

Esta definición se aplica también a L, considerando a G como su conjunto de axiomas

L es consistente

Por el absurdo

Suponemos que L no es consistente, entonces por definición de consistencia existe una fbf A tal que: $\vdash_L A$ y $\vdash_L (\neg A)$ Es decir, tanto A como $(\neg A)$ ambas son teoremas de L

Pero por la propiedad de corrección de L, sabemos que todos los teoremas de L son tautologías

Por lo tanto A es tautología (o sea $v(A)=V$ para toda valoración v) y al mismo tiempo $(\neg A)$ es tautología (o sea $v(\neg A)=V$), lo cual es absurdo pues $v(A) \neq v(\neg A)$

Decidibilidad

En la teoría de la computación, se define un problema de decisión como aquél que tiene dos respuestas posibles: “sí” o “no”

Se dice que un problema de decisión es decidible si existe un algoritmo (es decir, un procedimiento que siempre termina) que lo resuelve

En el marco de los sistemas deductivos es muy relevante también la cuestión de la decidibilidad, que se formula de la siguiente manera: dados Γ y A , ¿existe algún algoritmo que responda “sí” en el caso de que $\Gamma \models A$ y que responda “no” en el caso de que $\Gamma \not\models A$?

Claramente, en la lógica proposicional esta cuestión tiene una respuesta positiva, utilizando algoritmos basados en las tablas de verdad

Clase 11 - Lógica de predicados

La lógica proposicional permite formalizar y teorizar sobre la validez de una gran cantidad de enunciados

Sin embargo existen enunciados intuitivamente válidos que no pueden ser probados por dicha lógica

Por ejemplo, considérese el siguiente razonamiento:

Todos los hombres son mortales.
Sócrates es un hombre.
Por lo tanto, Sócrates es mortal.

Su formalización es la siguiente:

p
q
Por lo tanto, r

Esta es claramente una forma de razonamiento inválido (ya que p y q pueden ser V, y r ser F), lo que contradice nuestra intuición

Dicha problemática la resuelve la lógica de predicados

Esto se corresponde a las ideas de:

- Sujetos y Predicados
- Cuantificadores

El Sujeto es la cosa acerca de la cual el enunciado está afirmando algo, ej. Sócrates

El Predicado se refiere a una propiedad que posee el sujeto, ej «es mortal»

Predicados

Un predicado es “lo que se afirma de un sujeto en una proposición” (D.R.A.E.)

Los predicados pueden definir propiedades sobre uno, dos, o más individuos (u objetos), establecen relaciones entre ellos

Así, hay predicados unarios (o monádicos), binarios, ternarios,...,n-arios

Los predicados unarios definen relaciones de grado uno, es decir propiedades de un objeto, como por ejemplo: “el 7 es un número primo” que lo simbolizamos $P_1^1(7)$

Los predicados binarios definen una relación de grado dos, por ejemplo: “9 es múltiplo de 3” que lo simbolizamos $P_1^2(9,3)$

Cuantificadores

Usando el lenguaje simbólico que acabamos de introducir, podemos escribir “Todo entero tiene un factor primo” como:

Para todo x , $E(x) \rightarrow P(x)$ Donde $E(x)$ simboliza « x es un entero» y $P(x)$ simboliza « x tiene un factor primo»

Introduciendo un símbolo para el cuantificador:

$$(\forall x) (E(x) \rightarrow P(x))$$

La frase «Existe al menos un objeto x , tal que» se simboliza $(\exists x)$ y se denomina Cuantificador Existencial

Relación entre $\forall$ y $\exists$

«No todas las aves vuelan» $\neg(\forall x) (A(x) \rightarrow V(x))$

«Algunas aves no vuelan» $(\exists x) (A(x) \wedge \neg V(x))$

El cuantificador universal va seguido de una implicación, debido a que los enunciados universales suelen ser de la forma, «dado un x cualquiera, si tiene la propiedad A entonces tiene también la propiedad B»

El cuantificador existencial va seguido de una conjunción, debido a que los enunciados existenciales suelen ser de la forma, «existe al menos un x , que tiene la propiedad A y tiene también la propiedad B»

Sintaxis

El lenguaje simbólico de la lógica

Para estudiar los principios del razonamiento, la lógica necesita en primer término capturar y formalizar las estructuras del lenguaje natural en un lenguaje simbólico, para luego formalizar los mecanismos de razonamiento que se aplican sobre dichas estructuras lingüísticas

El lenguaje simbólico consta de:

- El alfabeto o vocabulario: es el conjunto de símbolos primitivos que pertenecen al lenguaje
- La gramática: consiste en un conjunto de reglas que definen recursivamente las cadenas de símbolos que pertenecen al lenguaje

Alfabeto

El alfabeto del lenguaje está formado por:

- Un conjunto de símbolos de constantes $C = \{c_1, c_2, \dots\}$
- Un conjunto de símbolos de variables $X = \{x_1, x_2, \dots\}$
- Un conjunto de símbolos de funciones $F = \{f_1^1, f_2^1, \dots, f_1^2, f_2^2, \dots\}$
- Un conjunto de símbolos de predicados $P = \{P_1^1, P_2^1, \dots, P_1^2, P_2^2, \dots\}$
- Símbolos de conectivas (los mismos de la lógica proposicional): $\neg, \wedge, \vee, \rightarrow, \leftrightarrow$
- Paréntesis de apertura y cierre
- El cuantificador universal $\forall$ ("para todo") y el cuantificador existencial $\exists$ ("existe")

Gramática

La gramática del lenguaje define dos clases de elementos, por un lado los términos, que son las expresiones que denotan los objetos del dominio, y por el otro las fórmulas bien formadas (fbf), con las que se expresan las relaciones entre los objetos

Los términos se definen inductivamente de la siguiente manera:

- i. Los símbolos de constantes y de variables son términos
- ii. Si $t_1, \dots, t_n$ son términos y f_i^n es un símbolo de función, entonces $f_i^n(t_1, \dots, t_n)$ es un término
- iii. Sólo las expresiones que pueden ser generadas mediante las cláusulas i y ii en un número finito de pasos son términos

Por su parte, las fórmulas bien formadas se definen así:

- i. Si $t_1, \dots, t_n$ son términos y P_i^n es un símbolo de predicado, entonces $P_i^n(t_1, \dots, t_n)$ es una fórmula bien formada. En este caso se denomina fórmula atómica o directamente átomo

- ii. Si A y B son fórmulas bien formadas, entonces $(\neg A)$, $(A \wedge B)$, $(A \vee B)$, $(A \rightarrow B)$ y $(A \leftrightarrow B)$ también lo son
- iii. Si A es una fórmula bien formada y x es un símbolo de variable, entonces $(\forall x) A$ y $(\exists x) A$ son fórmulas bien formadas
- iv. Sólo las expresiones que pueden ser generadas mediante las cláusulas i a iii en un número finito de pasos son fórmulas bien formadas

Scope de los cuantificadores

Los cuantificadores tienen un alcance o scope, o radio de acción determinado

Por ejemplo, en la fórmula bien formada: $(\forall x) (P_1^1(x) \rightarrow P_2^1(x))$ las dos ocurrencias del símbolo de variable x suceden dentro del alcance del cuantificador $\forall$

Mientras que en la fórmula bien formada: $(\forall x) P_1^1(x) \rightarrow P_2^1(x)$ el $\forall$ alcanza sólo a la primera ocurrencia de x

Diremos en el primer caso que x está ligada, y en el segundo que la primera x está ligada y la otra está libre (por lo que podría sustituirse por cualquier otro símbolo de variable)

Una fbf es abierta si contiene algún símbolo de variable libre, y cerrada si todos los símbolos de variables están ligados

Clase 12 - Lógica de predicados - Semántica

Dominio de interpretación

Para poder describir los aspectos semánticos de la lógica de predicados, necesitamos antes, introducir el concepto de dominio

Un dominio está constituido por:

- Un universo U , que es un conjunto no vacío de objetos
- Un conjunto finito F de funciones, cada una de las cuales asigna a una determinada cantidad de objetos o argumentos de U un objeto de U

$$F = \{\hat{f}_1^1, \hat{f}_2^1, \dots, \hat{f}_1^2, \hat{f}_2^2, \dots\}$$

El símbolo $\hat{f}_j^n$ denota la j -ésima función de n argumentos. $\hat{f}_i^n : U^n \rightarrow U$

- Un conjunto finito P de relaciones entre los objetos de U :

$$P = \{\hat{P}_1^1, \hat{P}_2^1, \dots, \hat{P}_1^2, \hat{P}_2^2, \dots\}$$

El símbolo $\hat{P}_j^n$ denota la j -ésima relación de grado n . $\hat{P}_i^n \subseteq U^n$

Ejemplo 1:

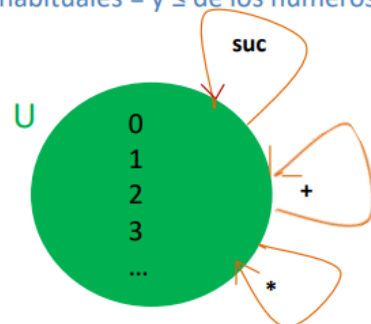
➤ $U = \mathbb{N}$, es decir que U es el conjunto de los números naturales.

➤ $F = \{\text{suc}, +, *\}$ o en general $F = \{f_1^1, f_1^2, f_2^2\}$

F es un conjunto con tres elementos, la función suc , la función suma y multiplicación ,
con $\text{suc} = \{(x, y) \mid y = x + 1\}$

➤ $P = \{=, \leq\}$ o en general $P = \{P_1^2, P_2^2\}$

P es el conjunto que tiene las relaciones habituales $=$ y $\leq$ de los números naturales,
que asumimos definidas.



Nota: las definiciones de las funciones y relaciones pueden ser extensionales, cuando se efectúan enumerando las tuplas que las componen, o intensionales, si se establecen a partir de otras funciones o relaciones

Ejemplo 2:

➤ $U = \{\text{Alex, Tomás, Catty, Florencia}\} \cup \mathbb{N}$. U es un conjunto de niños y números.

➤ $F = \{\text{edad, altura}\}$

siendo:

$\text{edad} = \{(\text{Alex}, 13), (\text{Tomás}, 15), (\text{Catty}, 14), (\text{Flor}, 15)\}$

$\text{altura} = \{(\text{Alex}, 174), (\text{Tomás}, 176), (\text{Catty}, 168), (\text{Flor}, 164)\}$

➤ $P = \{\text{juega-básquet, toca-piano, más-alto, más-joven, amigos}\}$

siendo:

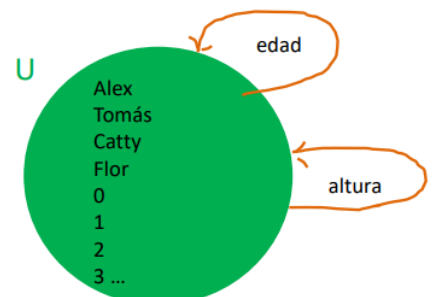
$\text{juega-básquet} = \{\text{Tomás}\}$

$\text{toca-piano} = \{\text{Alex, Catty}\}$

$\text{más-alto} = \{(x, y) \mid \text{altura}(x) > \text{altura}(y)\}$

$\text{más-joven} = \{(x, y) \mid \text{edad}(y) > \text{edad}(x)\}$

$\text{amigos} = \{(\text{Alex, Catty}), (\text{Alex, Flor}), (\text{Catty, Alex}), (\text{Flor, Tomás})\}$



Nota: obsérvese que las nociones de verdad y falsedad, en la lógica de predicados están directamente implícitas en el dominio. Por ejemplo: para establecer la verdad de que Alex toca el piano, incluimos a Alex en el conjunto de niños que tocan el piano, que es la manera extensional de definir dicha propiedad

Interpretación

Dados un lenguaje y un dominio, una interpretación I es una función que hace corresponder a los elementos del lenguaje con los elementos del dominio, satisfaciendo las siguientes condiciones:

- Si c_i es un símbolo de constante, entonces $I(c_i) \in U$ (los símbolos de constantes representan objetos del universo del discurso).
- Si f_i^n es un símbolo de función de grado n , entonces $I(f_i^n) = U^n \rightarrow U$ (los símbolos de función representan funciones del dominio).
- Si P_i^n es un símbolo de predicado de grado n , entonces $I(P_i^n) \subseteq U^n$ (los símbolos de predicado representan relaciones del dominio).

Por lo tanto, una interpretación formaliza la noción de que los símbolos representan las abstracciones de la realidad modelada en el dominio correspondiente

Ejemplo:

Dado un lenguaje con los símbolos c_1, f_1^1, f_1^2, f_2^2 y P_1^2

Lo interpretamos en un Dominio con:

Universo: $\mathbb{N}$

$F = \{+, \text{suc}, *\}$

$P = \{=\}$

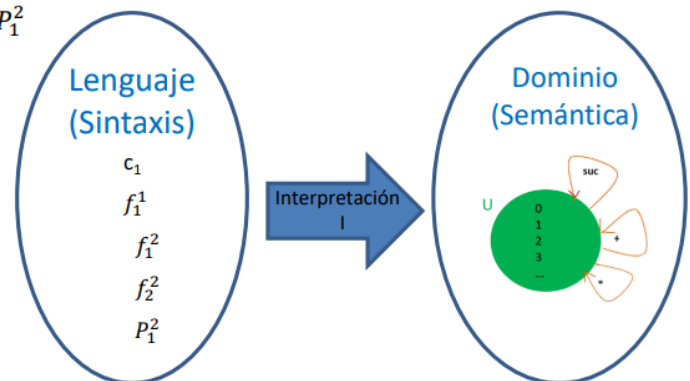
$I(c_1)$ es el cero de los naturales.

$I(f_1^1)$ es la función sucesor en los naturales.

$I(f_1^2)$ es la función suma en los naturales.

$I(f_2^2)$ es la función multiplicación.

$I(P_1^2)$ es la relación de igualdad.



Bajo esta interpretación sobre el dominio de los naturales:

$I(c_1)$ es el cero de los naturales.

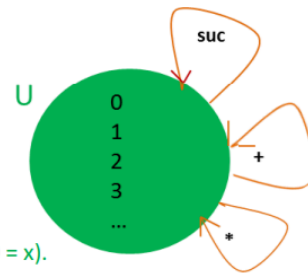
$I(f_1^2)$ es la función suma en los naturales.

$I(P_1^2)$ es la relación de igualdad.

Formulamos algunas propiedades :

$(\forall x) P_1^2(f_1^2(x, c_1), x)$. El cero es el neutro de la suma, es decir: $(\forall x)(x + 0 = x)$.

$(\forall x)(\forall y) P_1^2(f_1^2(x, y), f_1^2(y, x))$. La suma es conmutativa, es decir: $(\forall x)(\forall y)(x + y = y + x)$.



Si bien ésta es la interpretación “estándar”, nada, salvo el sentido común, nos impide plantear otras interpretaciones de los símbolos. Por ejemplo, podríamos establecer:

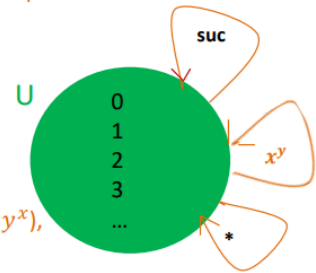
$I'(f_1^2)$ es la función potencia en los naturales, con potencia $(x, y) = x^y$

Bajo esta nueva interpretación, entonces, quedaría formulado lo siguiente:

$(\forall x) P_1^2(f_1^2(x, c_1), x)$. El cero es el neutro de la potencia, es decir: $(\forall x)(x^0 = x)$,

$(\forall x)(\forall y) P_1^2(f_1^2(x, y), f_1^2(y, x))$. La potencia es conmutativa, es decir: $(\forall x)(\forall y)(x^y = y^x)$,

Ninguna se cumple en el dominio considerado.



Observemos que:

- Los símbolos son sólo símbolos, carentes de «significado»
- Los símbolos obtienen un significado al ser interpretados sobre un Dominio Semántico
- El mismo símbolo puede interpretarse de distintas formas (no simultáneamente)

Semántica

Queda claro que si nos preguntamos acerca del «valor de verdad» de la siguiente fórmula:

$$(\forall x) P_1^2(f_1^2(x, c_1), x)$$

Nada podemos afirmar sin antes definir un dominio e interpretar el lenguaje

Para esta fórmula ya vimos dos interpretaciones diferentes:

1. En I , (Para todo número natural x) $(x + 0 = x)$
2. En I' (Para todo número natural x) $(x^0 = x)$

Analicemos el siguiente caso:

$$(\forall x) P_1^2(x, x) \rightarrow (\forall x) P_1^2(x, x)$$

¿Podemos afirmar cuál es su «valor de verdad» aun antes de interpretar el lenguaje?

Analicemos el siguiente caso:

$$(\forall x) P_1^2(y, x)$$

Bajo la interpretación sobre el dominio de los naturales con la relación $\leq$, (Para todo número natural x) $y \leq x$

Valoración

Para completar la definición anterior falta establecer una manera de asignar objetos a todos los términos del lenguaje

Esto se logra por medio de una función denominada valoración (en una interpretación)

Una valoración v , queda completamente especificada indicando cómo se valoran los símbolos de variables, es decir $v(x_1), v(x_2), \dots$, dado que se define:

- Si c_i es un símbolo de constante, $v(c_i) = I(c_i)$.
- Si f_i^n es un símbolo de función, $v(f_i^n(t_1, \dots, t_n)) = I(f_i^n)(v(t_1), \dots, v(t_n))$.

Por ejemplo, en la interpretación estándar de los naturales, fijando $v(x) = 7$, la valoración del término $f_1^1(f_1^2(x, c_1))$ se obtiene de la siguiente manera:

$$\begin{aligned} v(f_1^1(f_1^2(x, c_1))) &= I(f_1^1)(v(f_1^2(x, c_1))) = \text{suc}(v(f_1^2(x, c_1))) = \text{suc}(I(f_1^2)(v(x), v(c_1))) = \\ &= \text{suc}(+(v(x), v(c_1))) = \text{suc}(+(7, 0)) = \text{suc}(7) = 8 \end{aligned}$$

Satisfacción

Para denotar que una fórmula A se satisface con una interpretación I y una valoración v , escribiremos:

$$I \models_{I,v} A$$

La definición inductiva de la satisfacción de una fórmula es la siguiente:

$$\triangleright I \models_{I,v} P(t_1, \dots, t_n) \text{ si y sólo si } (v(t_1), \dots, v(t_n)) \in I(P)$$

Por ejemplo, sea I la interpretación en el dominio de los Naturales con la suma y la igualdad. Sea v , tal que $v(x)=5$, $v(y)=7$:

$$\begin{aligned} I \models_{I,v} P_1^2(f_1^2(x, c_1), x) &\text{ si y solo si } (v(f_1^2(x, c_1)), v(x)) \in I(P_1^2) \\ &\text{ si y solo si } (I(f_1^2)(v(x), v(c_1)), v(x)) \in I(P_1^2) \\ &\text{ si y solo si } (+ (5, 0), 5) \in I(P_1^2) \\ &\text{ si y solo si } (5, 5) \in = \\ &\text{ si y solo si } 5=5 \end{aligned}$$

Otro ejemplo:

$$\begin{aligned} I \models_{I,v} P_1^2(y, x) &\text{ si y solo si } (v(y), v(x)) \in I(P_1^2) \\ &\text{ si y solo si } (5, 7) \in = \\ &\text{ si y solo si } 5=7 \end{aligned}$$

A partir de la definición de satisfacción para una fbf atómica definimos inductivamente la noción de satisfacción para todas las fbfs del lenguaje:

Sean A y B fbfs del lenguaje:

- $\models_{i,v} (\neg A)$ si y sólo si no es el caso que $\models_{i,v} A$
- $\models_{i,v} (A \wedge B)$ si y sólo si $\models_{i,v} A$ y $\models_{i,v} B$
- $\models_{i,v} (A \rightarrow B)$ si y sólo si no es el caso que $\models_{i,v} A$ y no $\models_{i,v} B$
- $\models_{i,v} (\forall x_i) A$ si y sólo si para toda valoración w, i-equivalente, se cumple $\models_{i,w} A$
Es decir que A es verdadera cualquiera sea la valoración de x_i .
- $\models_{i,v} (\exists x) A$ si y sólo si para alguna valoración w, i-equivalente, se cumple $\models_{i,w} A$
En este caso alcanza con que A sea verdadera en alguna valoración particular de x.

i-equivalencias

Una valoración w es i-equivalente a otra valoración v, cuando $w(x_j) = v(x_j)$, para todo j tal que $j \neq i$:

	x1	x2	x3
v ₁	1	2	1
v ₂	3	2	1
v ₃	2	2	1

$v_1 v_2 v_3$ son 1-equivalente entre ellas

Verdad y validez

Si una fórmula es verdadera en toda interpretación es LÓGICAMENTE VÁLIDA

La notación es $\models A$

Por ejemplo, $((\forall x) P(x) \rightarrow (\exists x) P(x))$ es válida, cualquiera sea P.

Si una fórmula A, con una interpretación I, se satisface en todas las valoraciones, se dice que A es VERDADERA en I

Se escribe así: $\models_I A$

Por ejemplo, $(\forall x) P_1^2(f_1^2(x, c_1), x)$ es verdadera en la interpretación de los naturales con la suma y la igualdad.
 Pq todo número natural x cumple que $x + 0 = x$

Una fórmula A es SATISFACTIBLE cuando existe una interpretación I y una valoración v donde:

$$I \models_{I,v} A$$

Como contrapartida, decimos que una fórmula A es FALSA en una interpretación I si no existe ninguna valoración en I que la satisfaga

Por ejemplo, fórmula ya vista $(\forall x) P_1^2(f_1^2(x, c_1), x)$: todo numero natural x cumple que $x + 0 < x$ es falsa en la interpretación de los naturales con la suma y la relación menor.

Las fórmulas falsas en toda interpretación se identifican como CONTRADICCIONES
 Por ejemplo, la negación de cualquier fórmula lógicamente válida

Hay fórmulas que no son ni Verdaderas ni Falsas en una I

Por ejemplo: $P_1^2(x, y)$ si la interpretamos como $x < y$, se puede satisfacer cuando $v(x)=5$ y $v(y)=7$

Particularmente, una fórmula válida cuya estructura se corresponde con una tautología de la lógica proposicional, como por ejemplo $P(x) \vee (\neg P(x))$, es una tautología de la lógica de predicados

Las fórmulas válidas no proporcionan información alguna de un dominio

(a) Hemos visto que toda *fbf* de $\mathcal{L}$ que provenga de una tautología de L por sustitución es lógicamente válida (Proposición 3.31). Nótese, pues, que la clase de las *fbfs* de $\mathcal{L}$ lógicamente válidas contiene a la clase de las tautologías.

→ (b) $((\forall x_i) \mathcal{A} \rightarrow (\exists x_i) \mathcal{A})$ es lógicamente válida, cualquiera que sea la *fbf* $\mathcal{A}$ de $\mathcal{L}$. Esto se demuestra por un método standard como sigue.

Sea I una interpretación de dominio D_I y sea v una valoración en I . Si v no satisface $(\forall x_i) \mathcal{A}$ entonces v satisface $((\forall x_i) \mathcal{A} \rightarrow (\exists x_i) \mathcal{A})$. Si v satisface $(\forall x_i) \mathcal{A}$, entonces toda valoración v' que sea i -equivalente a v satisface $\mathcal{A}$. Está claro, entonces, que existe una valoración i -equivalente a v que satisface $\mathcal{A}$. Así pues, v satisface $(\exists x_i) \mathcal{A}$, por la Proposición 3.29. Así pues, también en este caso v satisface $((\forall x_i) \mathcal{A} \rightarrow (\exists x_i) \mathcal{A})$. Hemos demostrado, pues, que una valoración arbitraria en una interpretación arbitraria satisface la *fbf* dada, con lo que ésta es lógicamente válida.

(c) $((\forall x_1)(\exists x_2) A_1^2(x_1, x_2) \rightarrow (\exists x_1)(\forall x_2) A_1^2(x_1, x_2))$ no es lógicamente válida. La demostración quizá es algo menos inmediata, puesto que lo que tenemos que hacer es encontrar una interpretación en la que la *fbf* dada no sea verdadera. Hemos de elegir un dominio D_I , una interpretación para la letra de predicado A_1^2 , y una valoración que no satisfaga la *fbf*.

Clase 13 - Inteligencia artificial

Coloquialmente, el término inteligencia artificial se aplica cuando una máquina imita las funciones cognitivas que los humanos consideramos como propias de nuestras mentes, en especial: comunicarnos, resolver problemas, tomar decisiones y aprender

Existe en múltiples lugares cotidianos:

- Filtros de spam
- Predicción del clima
- Predicción de valores del mercado
- Búsquedas en la web
- Automóviles que se manejan solos
- Clasificación de imágenes
- NLP
- Text to speech
- Speech to text

Razones de éxito

¿Por qué la IA tiene éxito ahora y no en los 60's?

¿Por qué la IA avanzó así en los últimos 10 a 15 años?

¿Mejóro el hardware?

Sí!

Los procesadores modernos se han vuelto cada vez más potentes

La capacidad de distribuir el procesamiento entre varias computadoras ha mejorado la capacidad de analizar datos en tiempo récord

¿Se inventaron nuevas estructuras de datos?

Sí, pero no tanto...

Hay más conjuntos de datos disponibles para respaldar el análisis, (ej. datos meteorológicos, médicos, de redes sociales)

Muchos de estos datos están disponibles en la nube

El costo de almacenar y administrar los grandes datos ha bajado drásticamente

¿Se diseñaron nuevos algoritmos?

Sí, pero no tanto...

Se han puesto a disposición algoritmos de aprendizaje automático a través de comunidades de código abierto

¿Son IA todos los sistemas computacionales que imitan nuestras funciones cognitivas?

Es IA cuando la máquina no sigue un algoritmo (una receta) para resolver un problema, sino que encuentra su propia forma de resolverlo $\Rightarrow$ Machine Learning

Machine Learning (máquinas que aprenden)

ML se trata de generalizar comportamientos a partir de una información suministrada en forma de ejemplos

Es un proceso de inducción del conocimiento

Dado: training set $\{(x_i, y_i) \mid i = 1 \dots N\}$

Encontrar: una buena aproximación a $F: X \rightarrow Y$

Diseñaremos una inteligencia artificial para diagnosticar riesgo cardíaco

¿Cómo hacemos?

Le damos como entrada las historias clínicas de muchos pacientes con un diagnóstico ya definido por un experto humano (aprendizaje supervisado)

La máquina será capaz de aprender de esos datos y diagnosticar a nuevos pacientes que no estaban en los datos de entrenamiento

La IA logrará detectar patrones en los datos de entrada, ej.:

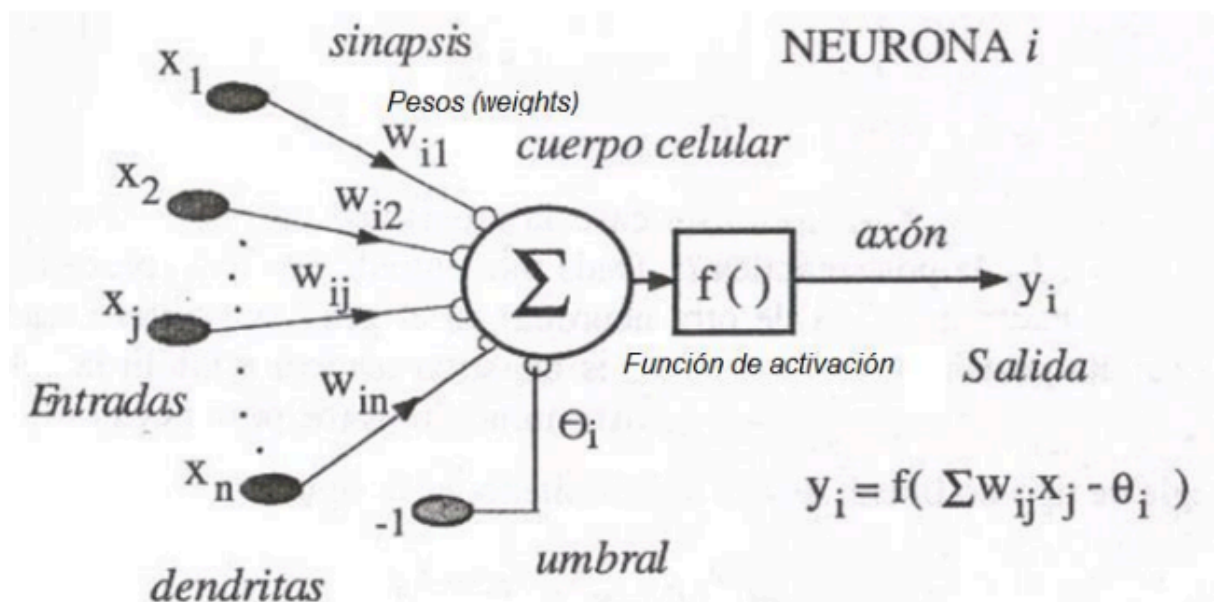
```
SI ((colesterol > 2.4 AND presión_arterial > 14
    AND glucosa > 1.20 AND edad > 50)
    OR (antecedentes_familiares AND sobrepeso AND tabaquismo ))
entonces riesgo_cardiaco = Verdadero
sino riesgo_cardiaco = Falso
```

Existen diferentes técnicas para implementar el Machine Learning
La más prometedora son las redes neuronales artificiales (ANN)

ML con ANN

Las redes neuronales son un modelo computacional basado en un gran conjunto de unidades neuronales simples (neuronas artificiales), de forma análoga a las neuronas en los cerebros biológicos

Cada unidad neuronal está conectada con muchas otras y los enlaces entre ellas pueden incrementar o inhibir el estado de activación de las neuronas adyacentes

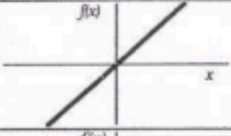
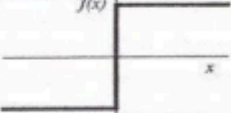
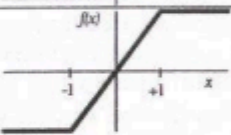
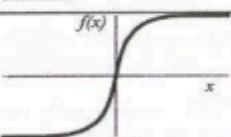
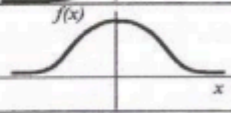
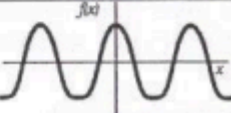


Las entradas tienen pesos (weights)

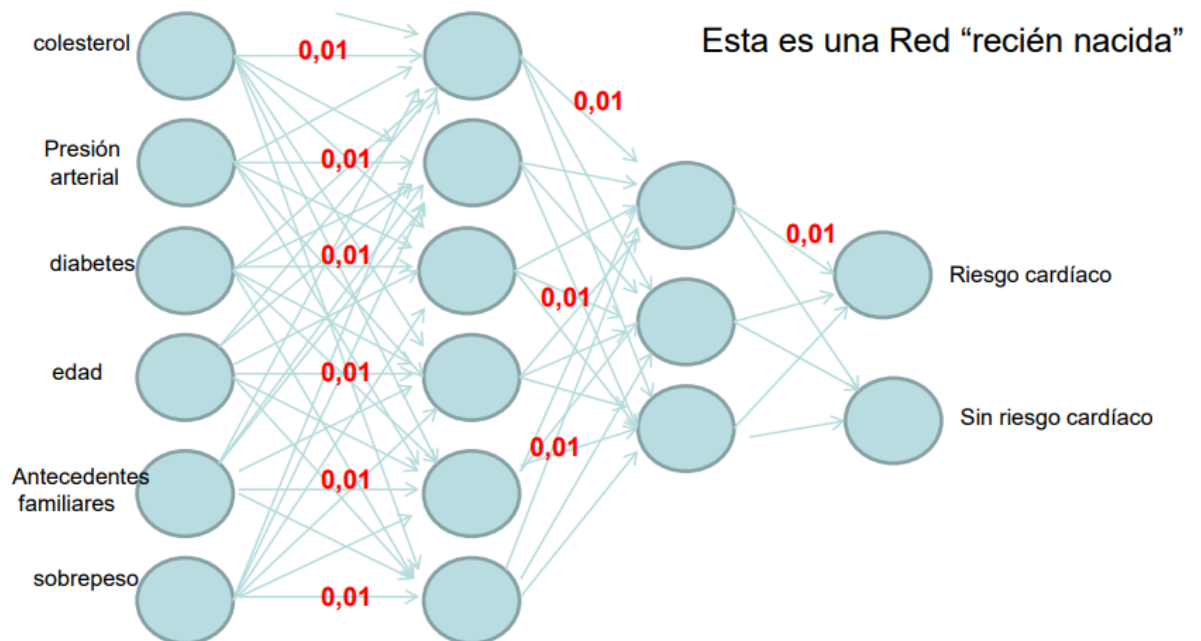
Cada unidad neuronal, de forma individual, opera empleando funciones de suma

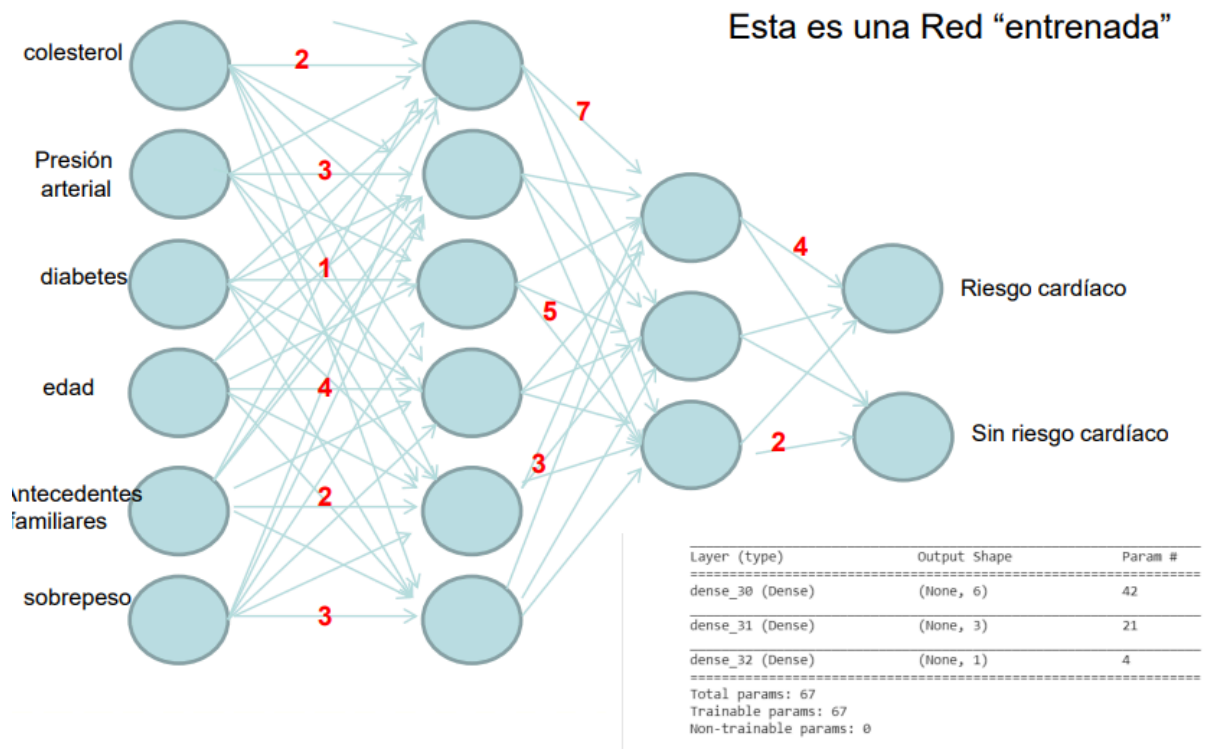
Existe una función limitadora o umbral, de tal modo que la señal debe sobrepasar un límite antes de propagarse a otra neurona

También aplica una función de activación $f()$

	Función	Rango	Gráfica
Identidad	$y = x$	$[-\infty, +\infty]$	
Escalón	$y = \text{sign}(x)$ $y = H(x)$	$\{-1, +1\}$ $\{0, +1\}$	
Lineal a tramos	$y = \begin{cases} -1, & \text{si } x < -l \\ x, & \text{si } -l \leq x \leq +l \\ +1, & \text{si } x > +l \end{cases}$	$[-1, +1]$	
Sigmoidea	$y = \frac{1}{1 + e^{-x}}$ $y = \text{tgh}(x)$	$[0, +1]$ $[-1, +1]$	
Gaussiana	$y = Ae^{-Bx^2}$	$[0, +1]$	
Sinusoidal	$y = A \text{sen}(\omega x + \varphi)$	$[-1, +1]$	

Las redes neuronales artificiales se crean e inician con valores aleatorios y luego se someten a un proceso de entrenamiento





El algoritmo de entrenamiento más utilizado se denomina retropropagación del error (En inglés, back-propagation)

En base a la derivada parcial de la función se puede saber si crece o decrece, es decir, hacia donde hay que ajustar el peso

Parámetros e hiper parámetros

Los parámetros del modelo son los pesos y los umbrales

El objetivo del entrenamiento es aprender los valores de estos parámetros

Los hiper parámetros son parámetros externos establecidos por el operador de la red neuronal

Hiper parámetros relacionados con la estructura de la red neuronal:

- Cantidad de capas ocultas ej.: 1
- Función de activación ej.: relu
- Inicialización de pesos ej.: valores chicos

Hiper parámetros relacionados con el entrenamiento:

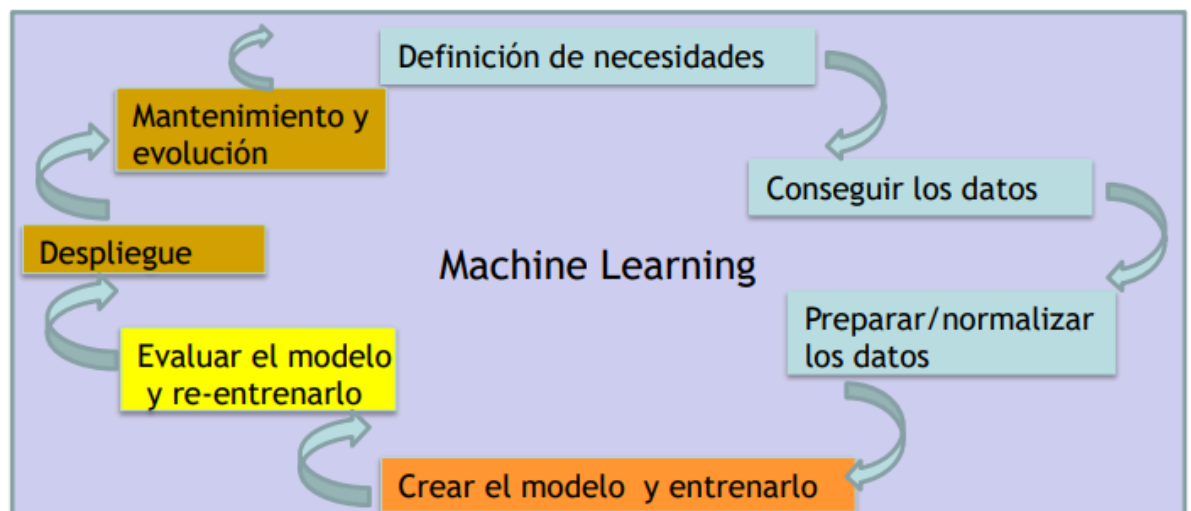
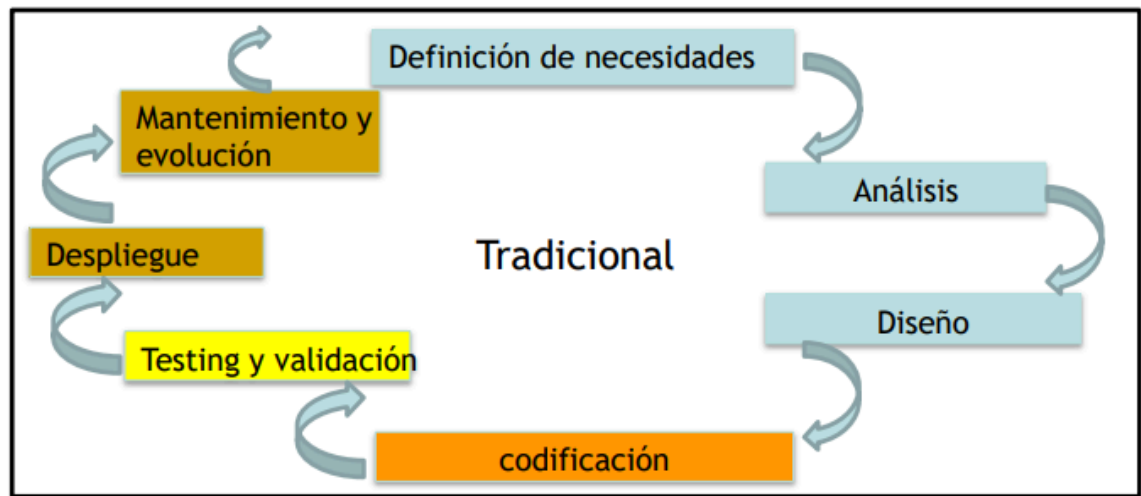
- Épocas
- Cantidad de iteraciones

- Tamaño de lote
- Algoritmo optimizador (ej. sgd descenso del gradiente, adam)
- Función de costo (o loss) ej. mse mean squared error

Métodos de ajuste de hiper parámetros

- Ajuste manual: un operador experimentado puede adivinar valores. Esto requiere prueba y error
- Búsqueda de cuadrícula: esto implica probar sistemáticamente múltiples valores de cada hiper parámetro y volver a entrenar el modelo para cada combinación
- Búsqueda aleatoria: un estudio de investigación mostró que el uso de valores de hiper parámetros aleatorios es en realidad más efectivo que la búsqueda manual o la búsqueda de cuadrícula
- Optimización bayesiana: métodos estadísticos han mostrado ser más efectivos en algunos dominios

Ciclo de vida del software tradicional vs ML



Conclusiones

Las máquinas nos sorprenden resolviendo problemas para los cuales nadie les escribió el algoritmo. Ellas por su cuenta son capaces de aprender el camino

Pero no son conscientes de lo que están haciendo, continúan siguiendo al pie de la letra otros algoritmos que los humanos hemos programado, los algoritmos de aprendizaje

La IA general se queda en Hollywood!! (se asocia con cualidades humanas como la conciencia, la sensibilidad, la sapiencia y el autoconocimiento)

Pero la IA aplicada está con nosotros!! (sólo pretende emular algunos aspectos concretos de la inteligencia humana)

La IA está siendo utilizada como herramienta de soporte a las decisiones en numerosas áreas (salud, educación, finanzas, seguridad, etc.), tanto en Argentina como en el resto del mundo

Permite la automatización de tareas que han requerido trabajo humano

La IA no es una caja mágica. Es un algoritmo matemático bien conocido

Su estructura no lineal anidada la hace poco transparente. La forma de entender el comportamiento de la IA es analizar los datos que se usaron para entrenarla

Los datos, son el combustible de la inteligencia artificial actual

Debería garantizarse que esos datos sean realmente “representativos”

Los datos tienen los mismos sesgos que existen en las sociedades

La IA no es un fin en sí misma, sino una herramienta para lograr otros fines. El objetivo es que la tecnología contribuya a mejorar el bienestar humano

Clase 14 - Verificación axiomática de programas

¿Cómo probar un programa?

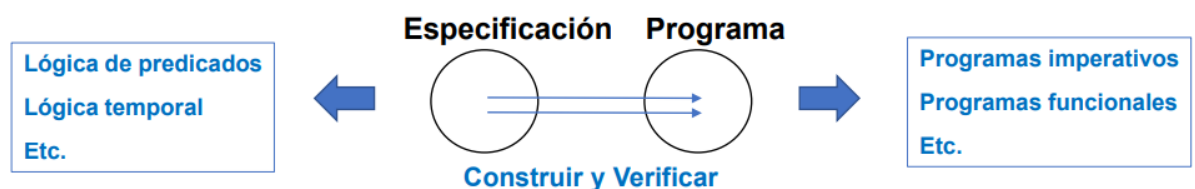
Verificación (actividad formal) vs validación (actividad informal, fundamentalmente el testing)

Dijkstra: “El testing asegura la presencia de errores pero no su ausencia”

Hoare: “Todas las propiedades de un programa pueden probarse en principio a partir de su propio texto por medio del puro razonamiento deductivo”

¿Programar y después verificar? ¿O programar y verificar simultáneamente?

Dijkstra: “Pensar cómo sería la prueba de un programa, y luego construirlo siguiendo la estructura de la prueba, para así construirlo y probarlo al mismo tiempo y obtener un programa correcto por construcción”



¿Cómo verificar un programa?

Ejemplo 1. Verificar el siguiente programa S_{swap} que permuta x con y :

Formalmente: $\{x = X \wedge y = Y\} S_{\text{swap}} \{y = X \wedge x = Y\}$

Ejemplo

$S_{\text{swap}}::$ $z := x;$ $x := y;$ $y := z$	$(x = 1, y = 2)$ $z := 1;$ $x := 2;$ $y := 1$
---	--

Dos maneras para hacerlo son:

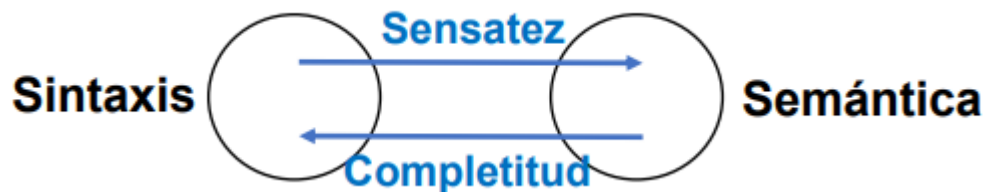
1) Por la vía semántica , usando la semántica de las instrucciones del lenguaje.	2) Por la vía sintáctica , usando axiomas y reglas de un método lógico-deductivo.																		
<table> <tr> <th>variables</th><th>programa</th></tr> <tr> <td>1) $x = X, y = Y$</td><td>$z := x; x := y; y := z$</td></tr> <tr> <td>2) $x = X, y = Y, z = X$</td><td>$x := y; y := z$</td></tr> <tr> <td>3) $x = Y, y = Y, z = X$</td><td>$y := z$</td></tr> <tr> <td>4) $x = Y, y = X, z = X$</td><td></td></tr> </table>	variables	programa	1) $x = X, y = Y$	$z := x; x := y; y := z$	2) $x = X, y = Y, z = X$	$x := y; y := z$	3) $x = Y, y = Y, z = X$	$y := z$	4) $x = Y, y = X, z = X$		<table> <tr> <td>1) $\{x = X \wedge y = Y\} z := x \{z = X \wedge y = Y\}$</td><td>ASI</td></tr> <tr> <td>2) $\{z = X \wedge y = Y\} x := y \{z = X \wedge x = Y\}$</td><td>ASI</td></tr> <tr> <td>3) $\{z = X \wedge x = Y\} y := z \{y = X \wedge x = Y\}$</td><td>ASI</td></tr> <tr> <td>4) $\{x = X \wedge y = Y\} z := x; x := y; y := z \{y = X \wedge x = Y\}$</td><td>SEC 1,2,3</td></tr> </table>	1) $\{x = X \wedge y = Y\} z := x \{z = X \wedge y = Y\}$	ASI	2) $\{z = X \wedge y = Y\} x := y \{z = X \wedge x = Y\}$	ASI	3) $\{z = X \wedge x = Y\} y := z \{y = X \wedge x = Y\}$	ASI	4) $\{x = X \wedge y = Y\} z := x; x := y; y := z \{y = X \wedge x = Y\}$	SEC 1,2,3
variables	programa																		
1) $x = X, y = Y$	$z := x; x := y; y := z$																		
2) $x = X, y = Y, z = X$	$x := y; y := z$																		
3) $x = Y, y = Y, z = X$	$y := z$																		
4) $x = Y, y = X, z = X$																			
1) $\{x = X \wedge y = Y\} z := x \{z = X \wedge y = Y\}$	ASI																		
2) $\{z = X \wedge y = Y\} x := y \{z = X \wedge x = Y\}$	ASI																		
3) $\{z = X \wedge x = Y\} y := z \{y = X \wedge x = Y\}$	ASI																		
4) $\{x = X \wedge y = Y\} z := x; x := y; y := z \{y = X \wedge x = Y\}$	SEC 1,2,3																		

La verificación semántica se complica mucho cuando los programas son complejos (sobre todo concurrentes)

Avanzaremos en lo que sigue con la verificación sintáctica (o axiomática), conocida como Lógica de Hoare

Lo básico que se exige de un método axiomático es que sea sensato: que no pruebe enunciados falsos

Es ideal que también sea completo: que pruebe todos los enunciados verdaderos



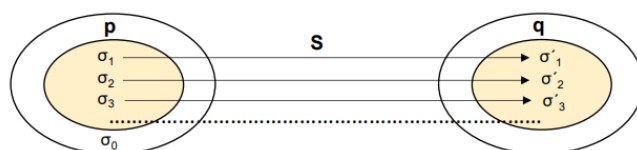
Definiciones

Volviendo al ejemplo del programa de swap:

- $\{x = X \wedge y = Y\} S_{\text{swap}} \{y = X \wedge x = Y\}$ es una **terna de Hoare** o **fórmula de correctitud**.
- El predicado $x = X \wedge y = Y$ es la **precondición** de S_{swap} .
- El predicado $y = X \wedge x = Y$ es la **postcondición** de S_{swap} .
- El par $(x = X \wedge y = Y, y = X \wedge x = Y)$ es la **especificación** de S_{swap} .
- Un **estado** σ es una función que asigna a toda variable un valor. Por ejemplo: $\sigma(x) = 1, \sigma(y) = 2$, etc.
- Un estado σ **satisface** un predicado p , si p evaluado con σ es verdadero. Se expresa así: $\sigma \models p$.
Por ejemplo: si $\sigma(x) = 1$ y $\sigma(y) = 2$, entonces $\sigma \models x < y$.

- Un programa S es **correcto con respecto a una especificación** (p, q) , lo que se expresa con $\{p\} S \{q\}$, sii:

Para todo estado σ , si $\sigma \models p$ entonces S ejecutado a partir de σ termina en un estado σ' tal que $\sigma' \models q$



P. ej., si $p = (x = X > 0)$ y $q = (y = 2.X)$, S debe hacer:
 Si $\sigma_1 \models x = 1$, entonces $\sigma'_1 \models y = 2$.
 Si $\sigma_2 \models x = 2$, entonces $\sigma'_2 \models y = 4$.
 Si $\sigma_3 \models x = 3$, entonces $\sigma'_3 \models y = 6$.
 Etc.

¿Se cumple $\{p\} S \{q\}$ si desde σ_0 , S no alcanza un σ'_0 dentro de q ? **Sí, porque σ_0 está fuera del alcance de p .**

Ejemplo 3. Verificar axiomáticamente un programa S_{fac} que calcula el factorial:

- Definición: el factorial $x!$ de un número $x > 0$ es: $x! = 1.2.3 \dots x$. Por ejemplo, $4! = 1.2.3.4 = 24$.

- Sea:

```

 $S_{\text{fac}} ::$ 
 $a := 1 ; y := 1 ;$ 
while  $a < x$  do
   $a := a + 1 ; y := y \cdot a$ 
od

```

Ejemplo

```

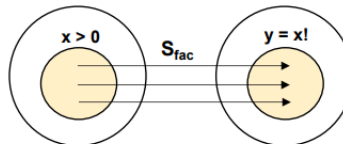
 $(x = 4)$ 
 $a = 1, y = 1$ 
 $1 < 4: a = 2, y = 1.2 = 2$ 
 $2 < 4: a = 3, y = 2.3 = 6$ 
 $3 < 4: a = 4, y = 6.4 = 24$ 

```

Se quiere verificar: $\{x > 0\} S_{\text{fac}} \{y = x!\}$

- Es decir, hay que probar que: desde todo estado $\sigma \models x > 0$, S_{fac} termina en un estado $\sigma' \models y = x!$

- Gráficamente:



Si $x = 1$, luego de S_{fac} debe valer $y = 1$.
 Si $x = 2$, luego de S_{fac} debe valer $y = 2$.
 Si $x = 3$, luego de S_{fac} debe valer $y = 6$.
 Etc.

Notar que si $x \leq 0$, queda $y = 1 \neq x!$ (no es correcto). ¿Esto significa que el programa es incorrecto?

NO. La precondition considera $x > 0$.

Componentes del método de verificación axiomática (Lógica de Hoare)

Lenguaje de programación (programas con while):

- Instrucciones:

$S :: x := e \mid S_1 ; S_2 \mid \text{if } B \text{ then } S_1 \text{ else } S_2 \text{ fi} \mid \text{while } B \text{ do } S_1 \text{ od}$

- Expresiones de tipo entero:

$e :: n \mid x \mid (e_1 + e_2) \mid (e_1 - e_2) \mid (e_1 \cdot e_2) \mid \dots$

n es una constante entera. x es una variable entera.

- Expresiones de tipo booleano

$B :: \text{true} \mid \text{false} \mid (e_1 = e_2) \mid (e_1 < e_2) \mid \dots \mid \neg B_1 \mid (B_1 \vee B_2) \mid (B_1 \wedge B_2) \mid \dots$

Ejemplo de programa

```
a := 1 ; y := 1 ;  
while a < x do  
    a := a + 1 ; y := y . a  
od
```

Lenguaje de especificación (lógica de predicados):

$p :: \text{true} \mid \text{false} \mid (e_1 = e_2) \mid (e_1 < e_2) \mid \dots \mid \neg p \mid (p_1 \vee p_2) \mid \dots \mid \exists x: p \mid \forall x: p$

Ejemplos de predicados

```
true  
x + 1 = y  
 $\neg(a < x)$   
 $\forall x: (x > y \vee x \leq y)$ 
```

Para simplificar, si se puede
se suprimen los paréntesis

Axiomática

Axioma de la asignación (ASI)

$\{p(e)\} x := e \{p(x)\}$

Si luego de $x := e$ vale p para x , entonces antes de $x := e$ valía p para e .

Por ejemplo: $\{y > 0\} x := y \{x > 0\}$

Ejercicio: $\{?\} x := x + 1 \{x > 0\}$ Rta: $\{x + 1 > 0\} x := x + 1 \{x > 0\}$

Regla de la secuencia (SEC)

$$\frac{\{p\} S_1 \{r\}, \{r\} S_2 \{q\}}{\{p\} S_1 ; S_2 \{q\}}$$

El predicado r actúa como nexos y luego se descarta, no se propaga

Regla del condicional (COND)

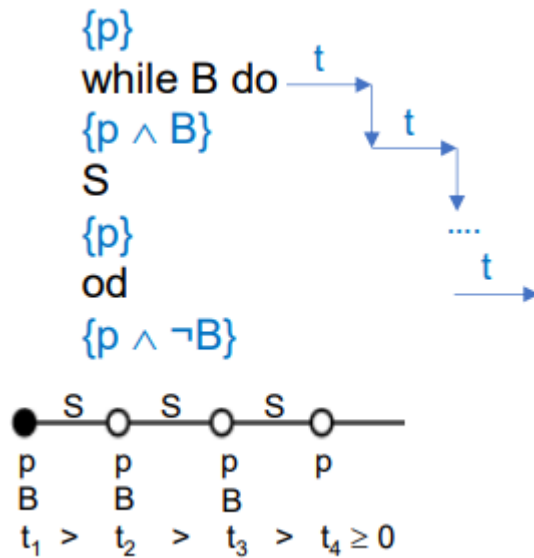
$$\frac{\{p \wedge B\} S_1 \{q\}, \{p \wedge \neg B\} S_2 \{q\}}{\{p\} \text{ if } B \text{ then } S_1 \text{ else } S_2 \text{ fi } \{q\}}$$

Formula un modo de verificar una selección condicional fijando un único punto de entrada y un único punto de salida, correspondientes a p y q, respectivamente

Regla de la repetición (REP)

$$\frac{\{p \wedge B\} S \{p\}, \{p \wedge B \wedge t = Z\} S \{t < Z\}, p \rightarrow t \geq 0}{\{p\} \text{ while } B \text{ do } S \text{ od } \{p \wedge \neg B\}}$$

p vale antes y después de toda iteración (invariante) t decrece después de toda iteración (variante)



Regla de consecuencia (CONS)

$$\frac{r \rightarrow p, \{p\} S \{q\}, q \rightarrow s}{\{r\} S \{s\}}$$

Permite reforzar precondiciones y debilitar postcondiciones

de $\{x > 0\} S \{x = 0\}$ y $(x > 5 \rightarrow x > 0)$ se deduce: $\{x > 5\} S \{x = 0\}$

de $\{\text{true}\} S \{x = y + 1\}$ y $(x = y + 1 \rightarrow x > y)$ se deduce: $\{\text{true}\} S \{x > y\}$

Composicionalidad

El método de prueba presentado es composicional:

Dado un programa S , compuesto por subprogramas $S_1, \dots, S_n$, que valga la fórmula $\{p\} S \{q\}$ depende sólo de que valgan fórmulas $\{p_1\} S_1 \{q_1\}, \dots, \{p_n\} S_n \{q_n\}$, sin importar el contenido de los S_i (noción de caja negra)

Por ejemplo, dado el programa $S :: S_1; S_2$, si se cumplen las fórmulas: $\{p\} S_1 \{r\}$ y $\{r\} S_2 \{q\}$, también se cumple la fórmula: $\{p\} S_1; S_2 \{q\}$, independientemente del contenido de S_1 y S_2

Más aún, si en lugar de S_2 utilizamos un subprograma S_3 que también satisface la fórmula: $\{r\} S_3 \{q\}$, entonces también se cumple la fórmula: $\{p\} S_1; S_3 \{q\}$, lo que significa que S_2 y S_3 son intercambiables (son funcionalmente equivalentes respecto de (r, q))

En los programas concurrentes la composicionalidad se pierde

Especificaciones

Hemos especificado un programa para calcular el factorial de la siguiente forma: $(x > 0, y = x!)$

¿Pero es correcta la especificación?

No

Por ejemplo, el programa $S :: x := 1 ; y := 1$ satisface $(x > 0, y = x!)$ pero no es el programa pedido:

$$\begin{array}{l} \{x = 5\} \\ x := 1 ; y := 1 \quad (\text{el resultado tiene que ser } y = 5! = 120) \\ \{y = 1\} \end{array}$$

Lo que sucede es que las variables de la precondition pueden modificarse a lo largo del programa

Lo que se hace es utilizar variables lógicas, para congelar valores

En el ejemplo considerado haríamos:

$$(x = X \wedge X > 0, y = X!)$$

¿Y se puede agregar a la especificación que la variable x no se modifique nunca?

No, la lógica de predicados no lo permite. Una lógica que sí lo permite es la lógica temporal

Sensatez, completitud y automatización de la verificación

Sensatez

Si se prueba $\{p\} S \{q\}$, se debe cumplir $\{p\} S \{q\}$

Propiedad obligatoria

Completitud

Si se cumple $\{p\} S \{q\}$, se debe poder probar $\{p\} S \{q\}$

Propiedad deseable

La completitud depende de la expresividad del lenguaje de especificación

Por ejemplo: dada la fórmula $\{p\} S1 ; S2 \{q\}$, debe poder encontrarse un predicado, tal que $\{p\} S1 \{r\} ; S2 \{q\}$

Automatización

La verificación de programas es en general indecidible, y por lo tanto no automatizable

Existen numerosas herramientas que asisten interactivamente al programador

Para los programas de estados finitos hay sistemas automáticos (model checking) con lógica temporal